Programación



3.02 · Entradas, salidas y conversiones · Python

Universidad Politécnica Salesiana · Ingeniería en Ciencias de la Computación · Cuenca
Unidad 3: Lenguaje de Programación
Docente: Ing. Pablo Torres, Mgtr.
Contacto: ptorresp@ups.edu.ec

Anterior: Introducción a Python · Siguiente: Estructuras de control

Descripción general

Un programa interactivo recibe datos, los procesa y comunica resultados. En este tema aprenderemos a utilizar `input()` y `print()`, convertir texto en números y construir mensajes mediante concatenación y f-strings. Trabajaremos con programas secuenciales: cada instrucción se ejecuta en el orden en que aparece.

Índice

- 1.1 Salida de datos con print()
- 1.2 Mensajes y formato con f-strings
- 1.3 Entrada de datos con input()
- 1.4 Conversión a texto con str()
- 1.5 Conversión a entero con int()
- 1.6 Conversión a decimal con float()
- 1.7 Significado, representación y errores frecuentes
- 1.8 Contrato de entrada y salida

1. Conceptos

1.1 Salida de datos con `print()`

Definición. `print()` muestra la representación textual de uno o varios valores. Puede recibir texto, números o expresiones.

Propósito. Comunicar información al usuario y observar resultados durante la ejecución.

```
print("Bienvenido a Programación")
print(10 + 5)
```

Salida:

```
Bienvenido a Programación
15
```

Por defecto, `print()` separa sus argumentos con un espacio y termina con un salto de línea. Los parámetros `sep` y `end` permiten cambiar esos separadores:

```
print("Código", "007", sep=": ")
print("Total: ", end="")
print(15)
```

Salida:

```
Código: 007
Total: 15
```

`print("Edad:", 18)` admite argumentos de distintos tipos. En cambio, la expresión `"Edad: " + 18` produce `TypeError`: el operador `+` no concatena directamente una cadena con un entero.

1.2 Mensajes y formato con f-strings

Definición. Una f-string comienza con `f` antes de las comillas. Las expresiones escritas entre llaves `{}` se evalúan y se incorporan al texto.

Propósito. Construir mensajes legibles que combinen etiquetas, variables y resultados.

```
nombre = "Ana"
edad = 20
print(f"Hola {nombre}, tu edad es {edad} años")
```

Salida:

```
Hola Ana, tu edad es 20 años
```

También podemos especificar cómo presentar un número:

```
importe = 7.0
print(f"Total: {importe:.2f}")
print(importe)
```

Salida:

```
Total: 7.00
7.0
```

En `:.2f`, el número `2` indica cuántas cifras se muestran después del punto decimal. El formato produce una representación redondeada; no modifica el valor guardado en `importe`.

1.3 Entrada de datos con `input()`

Definición. `input()` lee una línea de entrada y devuelve una cadena de tipo `str`. Elimina el salto de línea final, pero no elimina automáticamente los demás espacios.

Propósito. Obtener información que el usuario proporciona durante la ejecución.

```
nombre = input("Ingrese su nombre: ")
print(f"Hola, {nombre}")
```

Interacción de ejemplo: el usuario escribe `Ana` después del mensaje.

```
Ingrese su nombre: Ana
Hola, Ana
```

Aunque el usuario escriba `20`, el resultado de `input()` es el texto `"20"`. Para sumarle un valor numérico es necesario convertirlo.

Cada llamada a `input()` consume una línea. Una línea vacía produce `""`; si no queda ninguna línea disponible, se produce `EOFError`.

1.4 Conversión a texto con `str()`

Definición. `str()` obtiene una representación textual de un valor. Su uso no se limita a los enteros.

Propósito. Trabajar con un valor como texto; por ejemplo, concatenar una edad con una oración.

```
edad = 18
mensaje = "Tu edad es " + str(edad)
print(mensaje)
```

Salida:

```
Tu edad es 18
```

`str(edad)` devuelve una cadena nueva; la variable `edad` sigue almacenando el entero `18`. En una f-string no es necesario escribir esa conversión explícita: `f"Tu edad es {edad}"` construye el mensaje directamente.

1.5 Conversión a entero con `int()`

Definición. `int()` puede convertir una cadena que representa un número entero en un valor de tipo `int`.

Propósito. Realizar operaciones con cantidades enteras recibidas como texto.

```
numero_texto = "15"
numero = int(numero_texto)
print(numero + 5)
```

Salida:

```
20
```

Para capturar una cantidad por teclado se pueden escribir las dos operaciones juntas:

```
cantidad = int(input("Ingrese la cantidad: "))
print(f"Cantidad: {cantidad}")
```

`int("3.5")` produce `ValueError` porque la cadena no representa un entero. Aunque `int(3.5)` devuelve `3`, esa operación trunca hacia cero; no comprueba que el valor original fuera entero. No conviertas primero a `float` para ocultar una entrada incorrecta.

1.6 Conversión a decimal con `float()`

Definición. `float()` puede convertir una cadena numérica en un valor de punto flotante.

Propósito. Trabajar con mediciones, promedios y otras cantidades que pueden tener parte decimal.

```
precio_texto = "19.99"
precio = float(precio_texto)
print(f"Importe: {precio * 2:.2f}")
```

Salida:

```
Importe: 39.98
```

En las entradas de este tema se utiliza el punto como separador decimal: `"19.99"`. La cadena `"19,99"` produce `ValueError`; `float()` no sustituye la coma automáticamente.

Los valores `float` representan muchos decimales de forma aproximada. Usar `:.2f` mejora su presentación, pero no convierte la aritmética en decimal exacta. Aquí los importes sirven para practicar entrada, cálculo y salida.

1.7 Significado, representación y errores frecuentes

El tipo de un dato depende de su significado. Un código como `007` debe conservarse como texto si sus ceros iniciales forman parte del identificador. Convertirlo a entero produce `7`; volver a usar `str()` produce `"7"`, sin recuperar los ceros perdidos.

Expresión	Resultado	Interpretación
`"12" + "8"`	`"128"`	Concatenación de cadenas.
`int("12") + int("8")`	`20`	Suma de enteros.
`int("007")`	`7`	Se pierde el formato del código.
`int("3.5")`	`ValueError`	El texto no representa un entero.
`float("3.5")`	`3.5`	Conversión válida a punto flotante.
`float("3,5")`	`ValueError`	Separador decimal no admitido en este contrato.
`"Edad: " + 18`	`TypeError`	Tipos incompatibles para esta concatenación.
`bool("False")`	`True`	Una cadena no vacía se considera verdadera.

No uses `bool(input())` para interpretar respuestas como «sí», «no», «True» o «False». Reconocer esas palabras requiere una regla explícita que se trabajará con las estructuras de control.

Convertir no equivale a validar. `int("-2")` se convierte correctamente, aunque una cantidad negativa pueda estar prohibida por el problema. `float()` también acepta representaciones especiales como `"inf"` y `"nan"`; los ejercicios de este tema exigen números finitos y no incluyen esas entradas.

1.8 Contrato de entrada y salida

Antes de programar, especifica:

1. Qué datos se leen y en qué orden.
2. Qué tipo y dominio tiene cada dato.
3. Qué cálculo debe realizarse.
4. Qué texto y precisión debe tener cada resultado.
5. Qué ocurre si no se cumplen las condiciones de entrada.

En este tema los programas suponen entradas que cumplen el dominio indicado. No incorporan aún decisiones, bucles ni recuperación de errores con `try/except`. Una conversión incorrecta detiene el programa con una excepción; eso no debe describirse como una validación amigable.

Los ejemplos interactivos pueden mostrar mensajes en `input()`. En las demostraciones con salida exacta y archivos de prueba se usa `input()` sin mensaje para no añadir texto a la salida esperada.

2. Demostraciones

Lee el problema, predice el resultado y después ejecuta el código. Distingue siempre las líneas que escribe el usuario de las líneas que imprime el programa.

2.1 Texto y cálculo: resultados diferentes

Problema: observar cómo cambia una operación cuando sus operandos son cadenas o enteros.

```
a = "12"
b = "8"
print(a + b)
print(int(a) + int(b))
```

Salida:

```
128
20
```

La primera operación une caracteres. La segunda convierte ambos textos y suma los valores. Predice qué sucedería con `a = "5"` y `b = "7"` antes de repetir la ejecución.

2.2 Construcción de mensajes

Problema: presentar la misma edad usando concatenación, argumentos separados y una f-string.

```
edad = 18
print("Tu edad es " + str(edad))
print("Tu edad es", edad)
print(f"Tu edad es {edad}")
```

Salida:

```
Tu edad es 18
Tu edad es 18
Tu edad es 18
```

Las tres instrucciones presentan el mismo mensaje, pero construyen la salida de forma distinta. Explica por qué solo la primera necesita `str()` explícitamente.

2.3 Total de una compra

Problema: leer un producto, una cantidad y un precio unitario; mostrar el nombre y el total.

Contrato:

Línea de entrada	Dato	Tipo después de leer o convertir	Dominio
1	Producto	`str`	Nombre no vacío.
2	Cantidad	`int`	Entero no negativo.
3	Precio unitario	`float`	Número finito no negativo, con punto decimal si corresponde.

El total se calcula como `cantidad * precio` y se muestra con dos decimales. Las restricciones de dominio son precondiciones: este programa no comprueba nombres vacíos, valores negativos ni valores no finitos.

Código completo — `ejemplos/demo.py`:

```
producto = input()
cantidad = int(input())
precio = float(input())
print(f"Producto: {producto}")
print(f"Total: {cantidad * precio:.2f}")
```

Entrada:

```
Cuaderno
4
1.75
```

Salida esperada:

```
Producto: Cuaderno
Total: 7.00
```

Seguimiento del programa:

Instrucción	Estado o efecto
`producto = input()`	Guarda `"Cuaderno"`.

Instrucción	Estado o efecto
`cantidad = int(input())`	Convierte `"4"` al entero `4`.
`precio = float(input())`	Convierte `"1.75"` al valor `1.75`.
Primera llamada a `print()`	Muestra el nombre del producto.
Segunda llamada a `print()`	Calcula `7.0` y lo presenta como `7.00`.

2.4 Ejecución y pruebas reproducibles

Guarda el código en `ejemplos/demo.py` y abre la terminal en esa carpeta. En la organización del curso corresponde a `03/3.02-entradas-salidas/python/ejemplos/`.

En Linux o macOS:

```
python3 demo.py
```

En Windows, con el lanzador de Python instalado:

```
py demo.py
```

Escribe cada dato en una línea y pulsa Enter. El programa no muestra preguntas antes de leer porque la demostración define una salida exacta.

Para repetir la prueba, guarda las tres líneas de entrada en `entrada.txt`. En Bash o Zsh:

```
python3 demo.py < entrada.txt
```

En el símbolo del sistema de Windows (`cmd`):

```
py demo.py < entrada.txt
```

PowerShell no utiliza `<` de esta misma manera. Puedes ejecutar la versión interactiva anterior o invocar `cmd` desde PowerShell:

```
cmd /c "py demo.py < entrada.txt"
```

Casos para comprobar:

Entrada, en líneas separadas	Resultado o comportamiento esperado
`Cuaderno`, `4`, `1.75`	`Producto: Cuaderno` y `Total: 7.00`.
`Lápiz`, `0`, `0.50`	`Producto: Lápiz` y `Total: 0.00`.
`Carpeta azul`, `2`, `3.25`	Conserva el espacio del nombre y muestra `Total: 6.50`.
`Cuaderno`, `abc`, `1.75`	`ValueError` al convertir la cantidad; no llega a imprimir resultados.
`Cuaderno`, `4`, `1,75`	`ValueError` al convertir el precio.
Archivo con solo `Cuaderno` y `4`	`EOFError` al intentar leer la tercera línea.

Estos son resultados esperados para practicar; en tu evidencia registra lo que efectivamente obtuviste al ejecutar.

3. Ejercicios

Resuelve cada ejercicio en un archivo independiente. Usa una línea por dato y respeta el orden indicado. Para comparar salidas exactas, utiliza `input()` sin mensajes. Los datos proporcionados deben cumplir el dominio: en esta unidad no se pide implementar validaciones con estructuras de control.

En las tablas, `;` separa datos que se introducen en **líneas distintas**; no se escribe el punto y coma. No se incluyen soluciones completas.

3.1 Saludo personalizado

Lee un nombre no vacío y muestra un saludo con f-string. Conserva los espacios interiores del nombre.

Entrada	Salida esperada
`Ana`	`Hola, Ana`
`Ana María`	`Hola, Ana María`

3.2 Suma de enteros

Lee dos enteros, conviértelos con `int()` y muestra únicamente la suma.

Entrada	Salida esperada
`12`; `8`	`20`
`-4`; `4`	`0`

Explica por qué sumar directamente los resultados de dos llamadas a `input()` no resuelve este problema.

3.3 Nombre y edad

Lee un nombre y una edad entera no negativa. Presenta ambos valores mediante una f-string.

Entrada	Salida esperada
`Luis;19`	`Luis: 19 años`

3.4 Área de un triángulo

Lee base y altura como números finitos positivos. Calcula `area = (base * altura) / 2` y muestra el resultado con dos decimales.

Entrada	Salida esperada
`10.5;4`	`El área del triángulo es: 21.00`

3.5 Conversión de temperatura

Lee una temperatura Celsius finita y calcula Fahrenheit mediante `celsius * 9 / 5 + 32`. Presenta un decimal.

Entrada	Salida esperada
`25`	`Fahrenheit: 77.0`
`0`	`Fahrenheit: 32.0`

3.6 Código y cantidad

Lee un código no vacío como texto y una cantidad entera no negativa. Conserva los ceros iniciales del código.

Entrada	Salida esperada
`007;3`	`007: 3`

Justifica por qué código y cantidad requieren tipos distintos aunque ambos contengan dígitos.

3.7 División con formato

Lee un dividendo y un divisor finitos. El divisor debe ser distinto de cero. Muestra el cociente con tres decimales.

Entrada	Salida esperada
`7;2`	`3.500`
`1;3`	`0.333`

Explica qué error produce un divisor cero. No necesitas agregar todavía un mecanismo de recuperación.

3.8 Promedio de notas

Lee tres notas decimales entre 0 y 10. Calcula su promedio aritmético y muéstralo con dos decimales.

Entrada	Salida esperada
`8.5; 7.8; 9.2`	`El promedio es: 8.50`

3.9 Servicio con recargo

Lee horas y tarifa por hora, ambas finitas y no negativas. Calcula el subtotal, un recargo fijo del 10 % sobre ese subtotal y el total. Muestra cada importe con dos decimales.

Entrada: `3.5; 8`.

Salida esperada:

```
Subtotal: 28.00
Recargo: 2.80
Total: 30.80
```

3.10 Factura de dos productos

Lee el nombre, la cantidad entera y el precio unitario del primer producto; después, los mismos datos del segundo. Los nombres son no vacíos, las cantidades no negativas y los precios finitos no negativos.

Calcula los dos importes y el total. Usa instrucciones secuenciales, sin bucles. Todos los importes deben tener dos decimales.

Entrada: `A; 2; 3.5; B; 3; 2`.

Salida esperada:

```
A: 7.00
B: 6.00
Total: 13.00
```

3.11 Calculadora simple

Lee dos enteros y muestra suma, resta, multiplicación y división. El segundo entero debe ser distinto de cero. Utiliza f-strings y muestra la división sin especificar precisión, para observar la representación predeterminada.

Entrada: `5; 7`.

Salida esperada:

```
La suma es: 12
La resta es: -2
La multiplicación es: 35
La división es: 0.7142857142857143
```

3.12 Datos personales

Lee nombre, edad y ciudad, en ese orden. Nombre y ciudad son textos no vacíos; la edad es un entero no negativo. Presenta una oración mediante una f-string.

Entrada: `Carlos`; `21`; `Quito`.

Salida esperada:

```
Hola Carlos, tienes 21 años y vives en Quito.
```

3.13 Conversión de edad a texto

Define una variable entera `edad` con valor `19`. Conviértela mediante `str()` y construye la oración utilizando concatenación con `+`. Este ejercicio no requiere leer por teclado.

Salida esperada:

```
La edad del estudiante es 19 años.
```

Explica qué parte del programa cambiaría si usaras una f-string y por qué la conversión no modifica la variable original.

3.14 PC3.02 — Servicio con descuento

Problema. Construye un programa que calcule el importe de un servicio a partir de su nombre, las horas trabajadas, la tarifa por hora y un porcentaje de descuento.

Entradas y contrato

Línea	Dato	Dominio
1	Nombre del servicio	Texto no vacío.
2	Horas	Número finito no negativo.
3	Tarifa por hora	Número finito no negativo.
4	Porcentaje de descuento	Número finito entre 0 y 100, inclusive.

El descuento es la **cuarta línea de entrada** y el tercer dato numérico. Se aplica al subtotal, no a la tarifa antes de calcular las horas.

Calcula:

- Subtotal: horas multiplicadas por tarifa.
- Importe del descuento: subtotal multiplicado por el porcentaje y dividido entre 100.
- Total: subtotal menos importe del descuento.

Presenta el nombre y los tres importes, cada importe con dos decimales. Conserva los valores numéricos para los cálculos y aplica el formato al imprimir.

Casos de aceptación

Entrada, una línea por dato	Subtotal	Descuento	Total
`Tutoría; 4; 10; 25`	`40.00`	`10.00`	`30.00`
`Tutoría; 0; 10; 0`	`0.00`	`0.00`	`0.00`
`Tutoría; 2; 12.5; 100`	`25.00`	`25.00`	`0.00`
`Tutoría; 1.5; 10; 0`	`15.00`	`0.00`	`15.00`

Para el primer caso, la salida exacta es:

```
Servicio: Tutoría
Subtotal: 40.00
Descuento: 10.00
Total: 30.00
```

Como observación de error, ejecuta también una entrada con `abc` en la línea de horas. En Python debe documentarse el `ValueError` de la conversión. No se exige capturarlo. Un descuento fuera del rango es una violación del contrato, aunque `float()` pueda convertirlo.

Requisitos de entrega

- Implementa una sola versión, Python o Java. Esta guía desarrolla la versión Python; si eliges Java, documenta sus comandos y su comportamiento de error.
- Para Python, entrega `main.py` en una carpeta propia de la PC, separado de `ejemplos/demo.py`.
- Organiza el código en entrada, cálculo y salida mediante bloques claros. No se requieren funciones, decisiones, bucles ni `try/except`.
- Documenta tipos, orden de lectura, dominio y formato de salida.
- Conserva los casos de aceptación y agrega al menos un caso válido propio.
- No copies la demostración como solución: incorpora el descuento y los resultados pedidos. No se proporciona una solución completa de esta práctica.

Lo esencial

Herramienta	Uso principal	Precaución
`print()`	Mostrar valores.	La salida debe respetar el contrato.

Herramienta	Uso principal	Precaución
`input()`	Leer una línea como texto.	No convierte automáticamente a número.
f-string	Combinar texto, expresiones y formato.	Formatear no modifica el valor almacenado.
`str()`	Obtener texto a partir de un valor.	No recupera formatos perdidos, como ceros iniciales.
`int()`	Convertir texto entero a número.	`"3.5"` no es una cadena entera válida.
`float()`	Convertir texto numérico a punto flotante.	Usa punto decimal; convertir no valida el dominio.

Antes de ejecutar, identifica qué datos son texto, cuáles intervienen en cálculos y qué salida exacta debes producir.

Recursos y referencias

Documentación oficial

- Funciones incorporadas: input(), print(), str(), int() y float().
- Tutorial de Python: entrada y salida.
- Cadenas de texto formateadas.
- Aritmética de punto flotante: problemas y limitaciones.

Bibliografía de apoyo

- Python Software Foundation. *El tutorial de Python*.
- Lutz, M. *Learning Python*. O'Reilly Media.
- Matthes, E. *Python Crash Course*. No Starch Press.